Technology + Tech + Programming + Coding + Encryption +

Nodejs : How Password Hashing Algorithm Works ?

Piyali Das ❤️🤍💚 7 min read · Just now

“bcrypt doesn’t decrypt or extract the original password. The stored bcrypt string contains the salt and hashing parameters along with the derived hash. During login, bcrypt extracts those parameters, hashes the candidate password using the same salt and cost factor, and compares the resulting hash with the stored hash. If they match, the password is valid.”

```
                    REGISTRATION

User password
     |
     ▼
"Prince"
     |
     ▼
bcrypt.hash()
     |
     ├── generates random salt
     ├── uses cost factor
     └── derives password hash
```

```
      |
      ▼
Stored bcrypt string
      |
      ▼
MongoDB
```

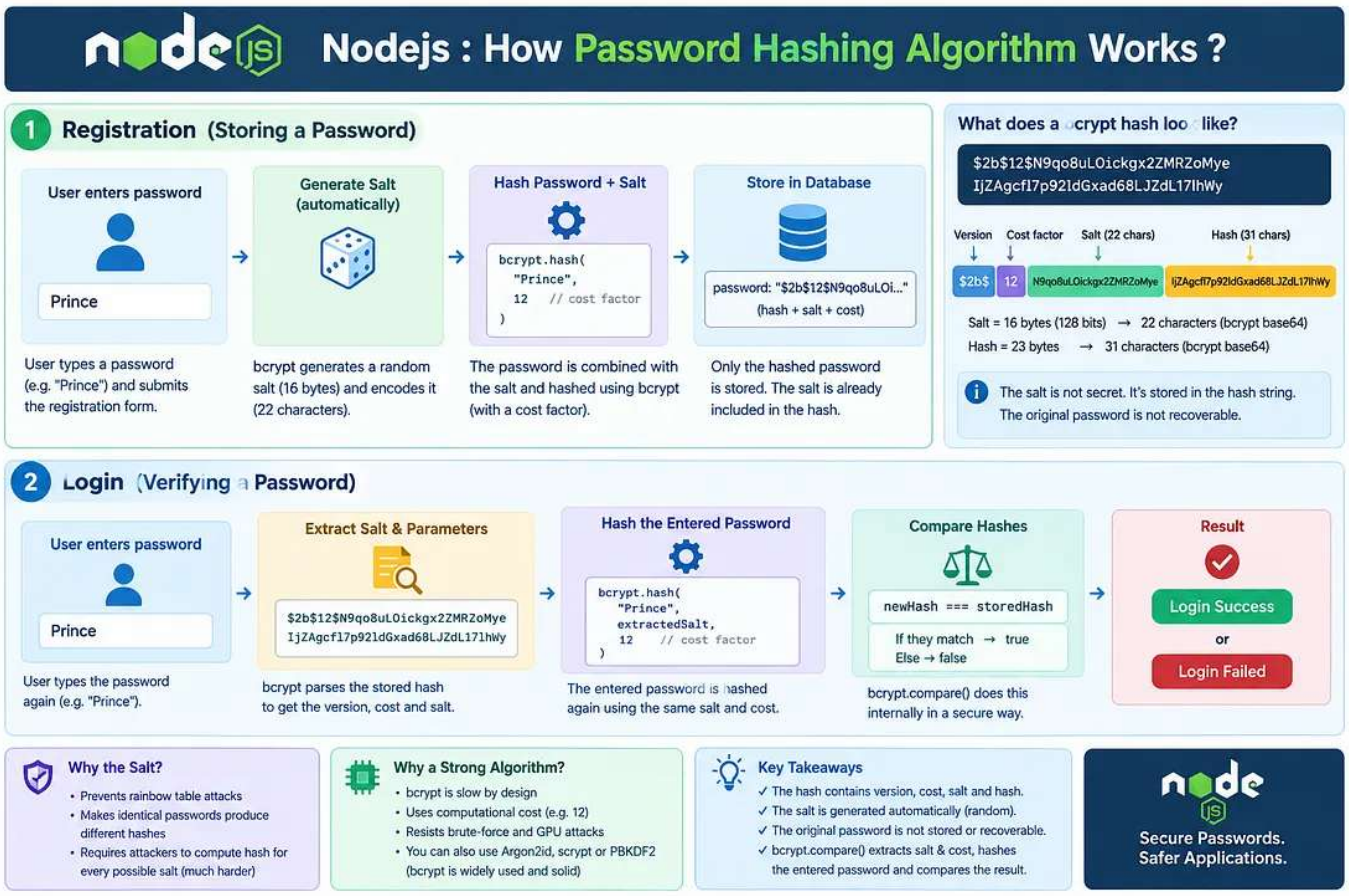
Then:

```
                        LOGIN

User enters
"Prince"
      |
      ▼
bcrypt.compare()
      ▲
      |
MongoDB
      |
Stored bcrypt string
      |
      ├── salt
      ├── cost
      └── stored hash
      |
      ▼
bcrypt calculates verification hash
      |
      ▼
Compare
      |
  ┌───┴────┐
  ▼        ▼
MATCH   NO MATCH
  |        |
  ▼        ▼
true     false
  |        |
  ▼        ▼
Login    Reject
```

The stored bcrypt hash does ***not contain the original password hidden inside it.***

It contains the **salt and hashing parameters**, plus the resulting hash. The original password cannot be extracted from it.



Node.js Password Hashing Explained

1. Registration

Suppose the user registers with:

```
Password = "Prince"
```

Your Mongoose `pre("save")` middleware does:

```
personSchema.pre("save", async function (next) {
  if (!this.isModified("password")) {
    return next();
  }

  this.password = await bcrypt.hash(this.password, 12);

  next();
});
```

Conceptually:

```
"Prince"
   |
   ▼
bcrypt generates random salt
   |
   ▼
bcrypt("Prince", salt, cost)
   |
   ▼
Stored bcrypt password
```

For example, a bcrypt value looks roughly like:

```
$2b$12$abcdefghijklmnopqrstuuXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

It encodes:

```
$2b$       → bcrypt version
12         → cost factor
```

```
salt       → salt information
hash       → derived password hash
```

So you don't normally need a separate:

```
salt: "..."
```

field when using bcrypt's standard encoded output.

2. Now the user logs in

The user enters:

```
Prince
```

Your database contains something like:

```
$2b$12$......................
```

The natural question is:

> **"How can bcrypt know whether `Prince` is correct when the database doesn't contain `Prince`?"**

This is the key concept.

You do:

```
const isMatch = await bcrypt.compare(
  candidatePassword,
  this.password
);
```

For example:

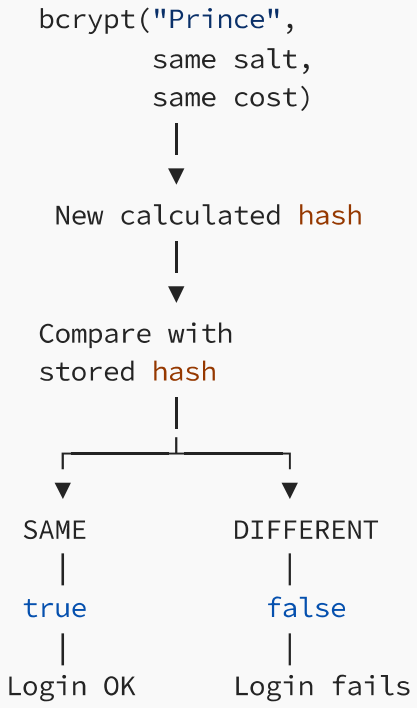
```
const isMatch = await bcrypt.compare(
  "Prince",
  "$2b$12$......................"
);
```

> *“bcrypt extracts the salt and cost information from the stored bcrypt string.”*

3. What does `bcrypt.compare()` actually do?

Conceptually:

```
DATABASE
                    |
                    ▼
        $2b$12$[salt][hash]
                    |
                    |
                    ▼
              Extract parameters
              + extract salt
                    |
                    ▼
User entered      "Prince"
                    |
                    ▼
```



If the password is correct

```
Stored:
bcrypt("Prince", stored-salt)
       ↓
       HASH_A

Entered:
"Prince"
   +
same stored salt
   ↓
bcrypt()
   ↓
HASH_A

HASH_A === HASH_A
       ↓
     true
```

If the password is wrong

User enters:

```
Princess
```

Then:

```
bcrypt("Princess", stored-salt)
       ↓
       HASH_B
```

But database contains:

```
HASH_A
```

Therefore:

```
HASH_B !== HASH_A
       ↓
     false
```

Login fails.

✅ Correct mental model

```
Stored bcrypt value
        |
        ├── algorithm/version
        ├── cost
        ├── salt
        └── password-derived hash
```

The password itself is **not recoverable.**

Then:

```
Entered password
        +
stored salt/parameters
        ↓
bcrypt calculation
        ↓
new hash
        ↓
compare with stored hash
```

That's why bcrypt is a **one-way password hashing mechanism**, not encryption.

How bcrypt salt and cost information contained in the stored bcrypt string ?

The important thing is that **bcrypt isn't doing magic or guessing the parameters.** The bcrypt output has a **defined format**, so the bcrypt library parses the string.

For example, a bcrypt hash commonly looks like:

```
$2b$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
```

Break it apart:

```
$2b$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
 |   |  |                       |
 |   |  |                       └── derived hash
 |   |  └── salt
 |   └── cost factor
 └── bcrypt version
```

More precisely:

```
$2b$12$[22-character salt][31-character bcrypt hash]
```

So what happens inside `bcrypt.compare()`?

When you do:

```
await bcrypt.compare("Prince", storedHash);
```

bcrypt essentially does:

```
storedHash
    |
```

```
    ▼
Parse the string
    │
    ├── $2b$  → algorithm/version
    │
    ├── 12    → cost factor
    │
    ├── salt → extract salt
    │
    └── hash → stored result
```

Then:

```
"Prince"
    +
extracted salt
    +
cost = 12
    │
    ▼
bcrypt algorithm
    │
    ▼
new hash
    │
    ▼
compare with stored hash
```

Think of it like a structured package

The stored string is not simply:

```
random_hash_string
```

It’s more like:

```
┌──────────────────────────────────────────────┐
│ $2b$ | 12 | SALT | DERIVED HASH              │
└──────────────────────────────────────────────┘
   |     |      |          |
   |     |      |          └── comparison target
   |     |      └── needed to reproduce hash
   |     └── computational cost
   └── bcrypt version
```

The library knows **where each piece is located because the bcrypt format is standardized.**

The security comes from:

```
Password + Salt + Expensive bcrypt computation
                          ↓
                        Hash
```

not from keeping the salt secret.

And this is why `bcrypt.compare()` is so useful

You don't have to manually do:

```
const salt = extractSalt(storedHash);

const newHash = bcrypt.hash(
  candidatePassword,
  salt
);
```

Instead:

```
const isMatch = await bcrypt.compare(
  candidatePassword,
  storedHash
);
```

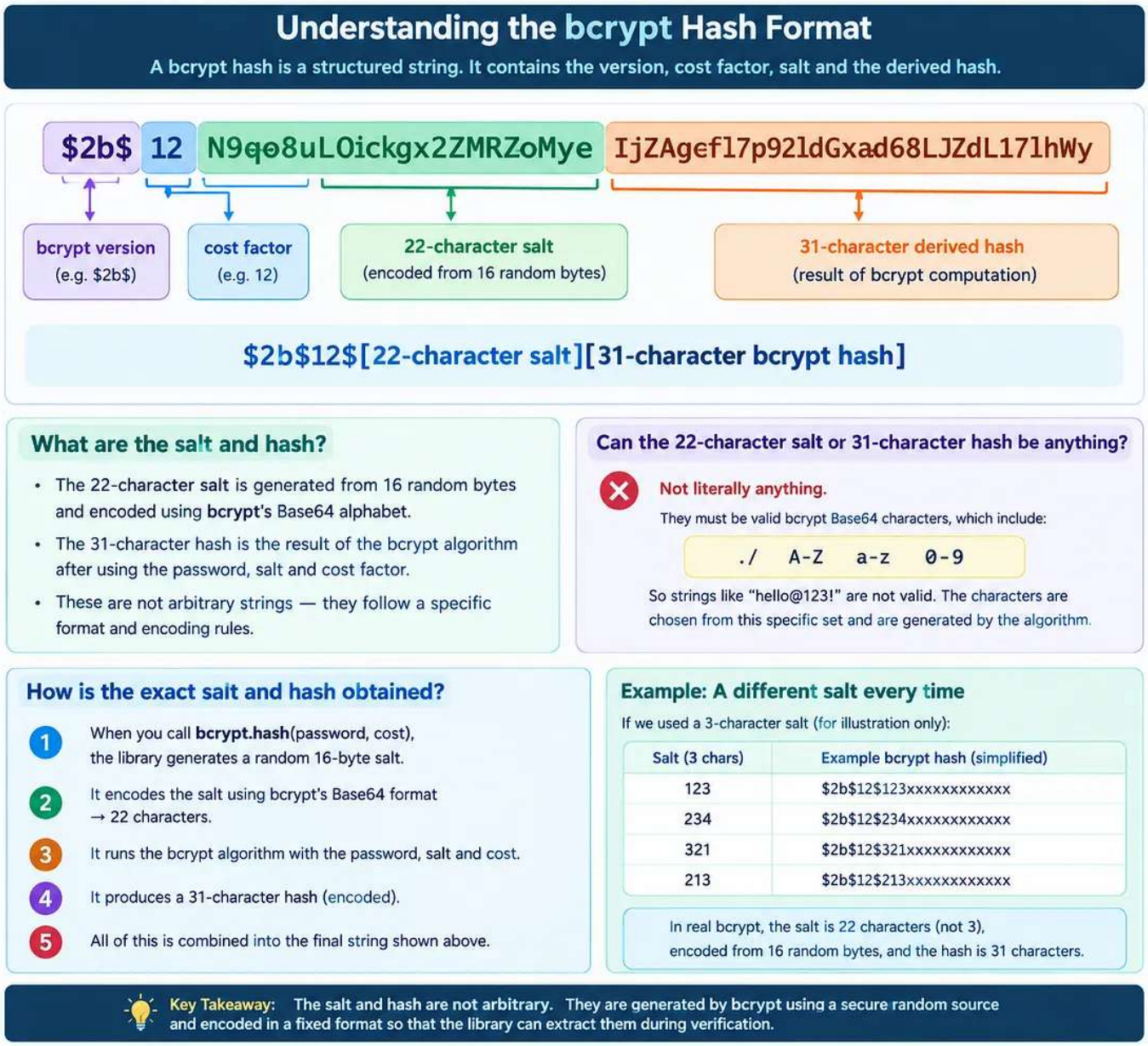
The bcrypt library handles the **parsing → salt extraction → cost extraction → hashing → comparison** internally.

> *"A bcrypt hash is a structured encoded string containing the bcrypt version, cost factor, salt, and derived hash.* `bcrypt.compare()` *parses that string, retrieves the salt and cost, hashes the candidate password with those same parameters, and performs a constant-time comparison against the stored hash."*

How bcrypt will get the exact salt & bcrypt hash ?

```
$2b$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
 | | | |
 | | | └── derived hash
 | | └── salt
 | └── cost factor
 └── bcrypt version
```

$2b$12$[22-character salt][31-character bcrypt hash]

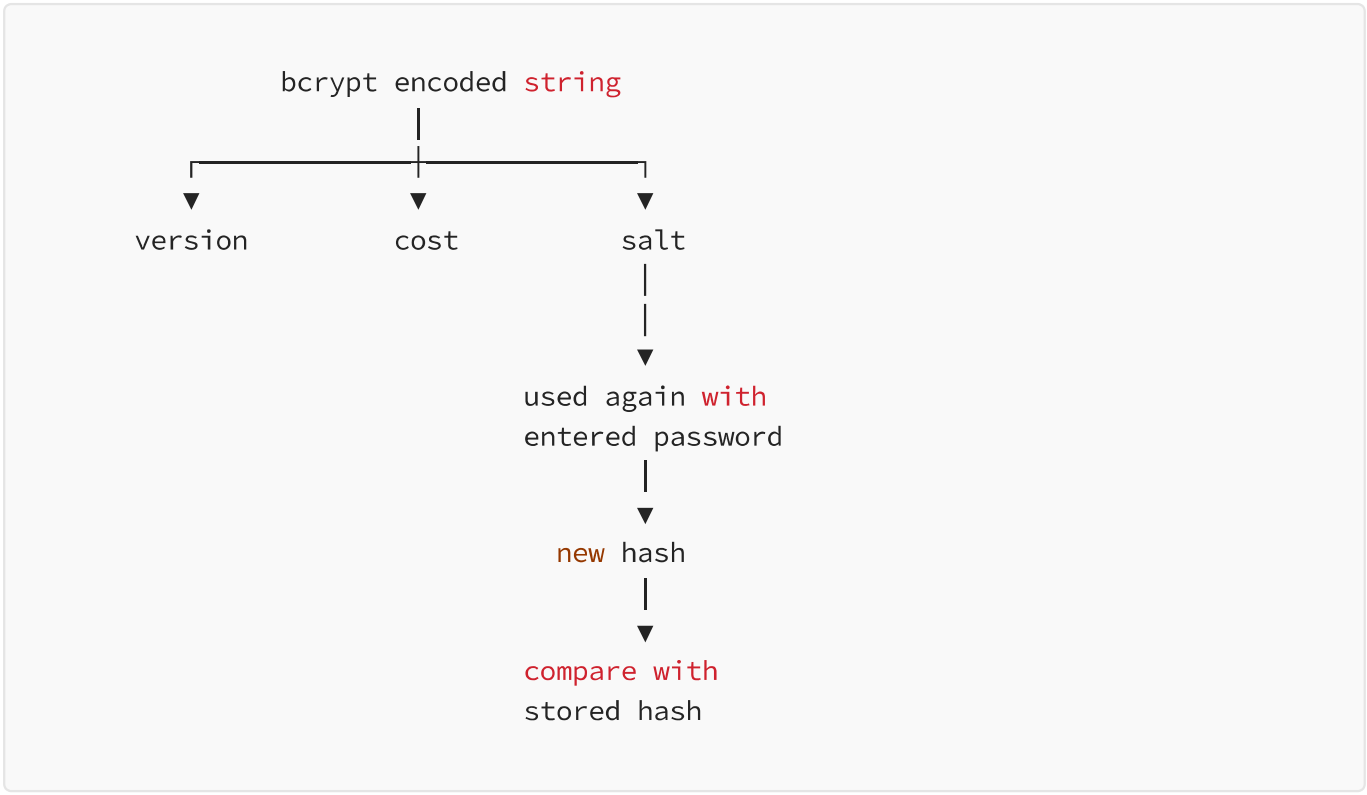


bcrypt Hash Format

22-character salt or 31-character bcrypt hash could be any ? how will get the exact salt & bcrypt hash ? Suppose 3-character salt could be 123, 234, 321, 213 etc.

Ans. *The salt and hash are not arbitrary strings chosen by the developer. They are generated/encoded according to bcrypt's format.*

Think:



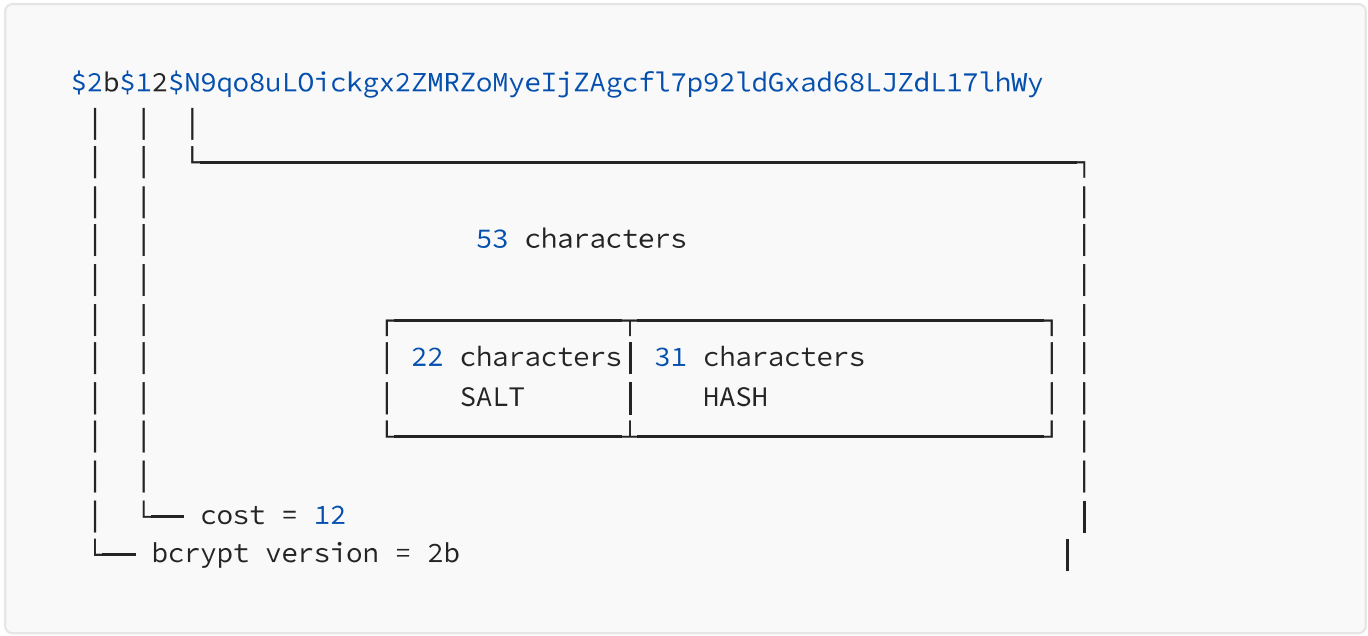
And your question about `123`, `234`, `321`, `213` is actually getting to the heart of **why a salt works:**

The salt is random, unique per password, and public. The exact salt is preserved in the stored bcrypt string so bcrypt can reproduce the same password-derived result during verification.

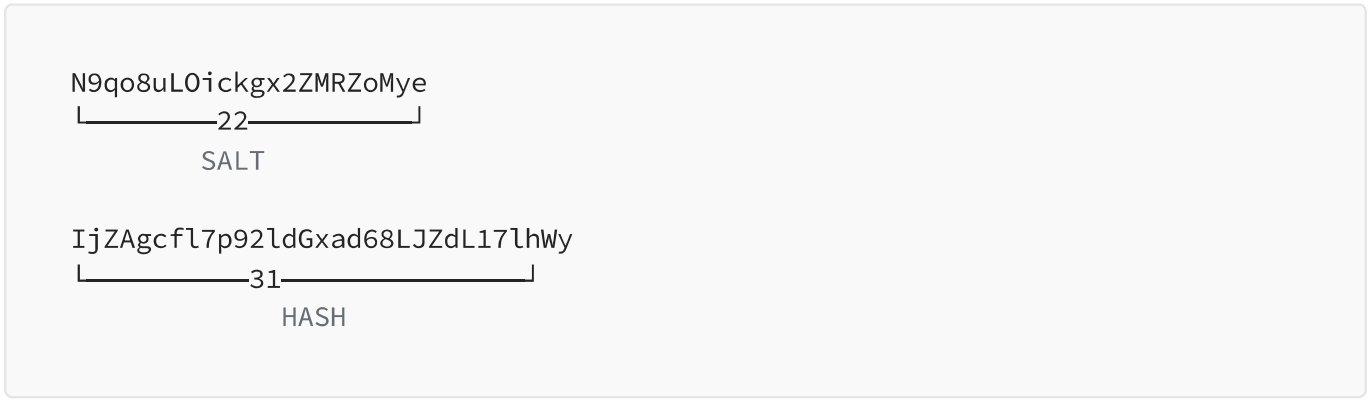
Let's take your example:

```
$2b$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
```

1. First split it



So:



2. Can the 22 salt characters be anything?

Not literally anything.

bcrypt uses a specific encoding alphabet:

```
. / A-Z a-z 0-9
```

So characters such as:

```
N9qo8uLOickgx2ZMRZoMye
```

are valid bcrypt-encoded characters.

But something like:

```
hello@123!XYZ
```

isn't a valid bcrypt salt representation.

More importantly, the 22 characters represent **16 random bytes (128 bits)** encoded using bcrypt's Base64-like encoding.

So conceptually:

```
Cryptographically random bytes
          ↓
16 bytes
          ↓
bcrypt Base64 encoding
          ↓
22 characters
```

For example, bcrypt might generate:

```
Random bytes
      ↓
N9qo8uLOickgx2ZMRZoMye
```

Another password registration could get:

```
Xf7KpQ2Lm9AaBcDeR4TyUv
```

and another:

```
3qWz8LmN0PabcXYZ...
```

The exact value is random.

3. Your 3-character example

You said:

> Suppose 3-character salt could be `123`, `234`, `321`, `213` etc.

Conceptually, **yes**, if we were designing our own toy system, a salt could be a random value like:

```
123
234
```

```
321
213
```

But bcrypt does **not** use a 3-character salt.

bcrypt uses a **16-byte random salt**, which is represented as **22 bcrypt-base64 characters**.

So think:

```
Toy example:

Salt = 123

Real bcrypt:
Random 128-bit salt
       ↓
bcrypt encoding
       ↓
22-character representation
```

4. But where does the exact salt come from?

When you execute:

```
const hash = await bcrypt.hash("Prince", 12);
```

you didn't provide a salt.

bcrypt internally does something conceptually like:

So **bcrypt itself generates the salt** using a cryptographically secure random source provided by the underlying platform/library.

5. Now the really interesting part: the 31-character hash

This part is completely different from the salt.

The salt is:

```
RANDOM
```

The hash is:

```
DETERMINISTIC
```

Suppose:

```
Password = Prince
Salt = ABCDEFGHIJKLMNOPQRSTUV
Cost = 12
```

bcrypt performs its expensive password derivation:

```
Prince
   +
Salt
   +
Cost 12
   ↓
bcrypt algorithm
   ↓
31-character encoded hash
```

For the **same password + same salt + same cost**, you get the **same result.**

So:

```
bcrypt("Prince", salt-A, 12)
        ↓
Hash-A
```

Run it again:

```
bcrypt("Prince", salt-A, 12)
        ↓
Hash-A
```

Same result.

But change the password:

```
bcrypt("Princess", salt-A, 12)
        ↓
Hash-B
```

Now:

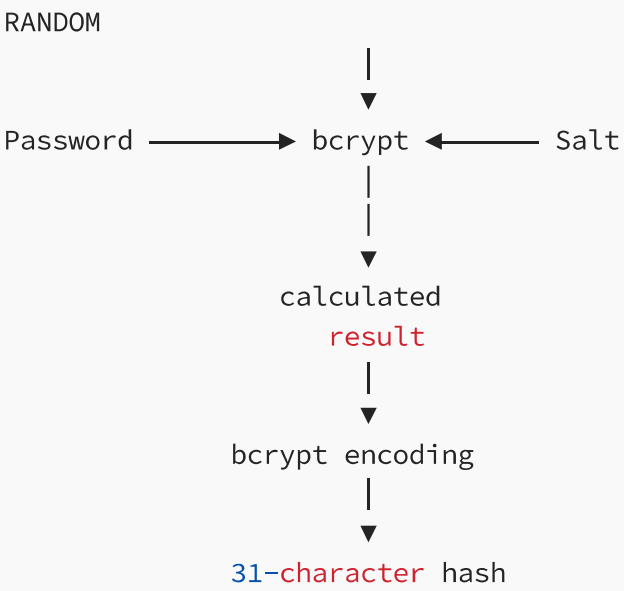
```
Hash-A !== Hash-B
```

6. So who “chooses” the 31 characters?

Nobody chooses them manually.

They are the **encoded result of the bcrypt calculation.**

Think of it like this:



Therefore:

```
22-character salt
       ↓
generated from random data

31-character hash
       ↓
generated from bcrypt computation
```

That's a very important distinction.

7. Why exactly 22 + 31?

bcrypt's encoded format has fixed fields.

For a normal bcrypt password hash:

```
$2b$12$ssssssssssssssssssssshhhhhhhhhhhhhhhhhhhhhhhhhhh
       └─────22─────┘└──────────31──────────┘
              salt                hash
```

The underlying values are approximately:

```
Salt:
16 bytes = 128 bits
        ↓
22 bcrypt-base64 characters

Hash:
23 bytes of bcrypt output
        ↓
31 bcrypt-base64 characters
```

The encoding isn't ordinary RFC 4648 Base64; bcrypt has its own Base64 alphabet/format.

8. Now connect this back to `compare()`

This is where everything becomes clear.

Database:

```
$2b$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
       └─────22─────┘└──────────31──────────┘
             SALT              HASH
```

User enters:

```
Prince
```

Then:

```
bcrypt.compare(
    "Prince",
    storedHash
)
```

bcrypt reads:

```
$2b$
 ↓
version
12
 ↓
cost
N9qo8uLOickgx2ZMRZoMye
 ↓
salt
IjZAgcfl7p92ldGxad68LJZdL17lhWy
 ↓
stored hash
```

Then it effectively calculates:

```
"Prince"
    +
stored salt
    +
cost 12
```

```
    ↓
bcrypt
    ↓
new calculated hash
```

And compares:

```
New calculated hash
        ==
Stored hash
```

If:

```
YES → true
NO  → false
```

Technology + Tech + Programming + Coding + Encryption +

Written by Piyali Das ❤️ 🤍 💚

818 followers · 9 following

Senior Frontend Engineer | Angular Expert | AI Enthusiast
https://www.linkedin.com/in/piyalidas10/

Follow